Cross-Platform Mobile Systems Architecture: Integrating a Shared Rust Core with Native UI and Continuous Biometric Data Collection

Executive Synthesis

The convergence of continuous biometric data ingestion, digital signal processing (DSP), and cross-platform mobile deployment represents a uniquely demanding domain in modern software engineering. Transitioning platform-specific business logic—currently implemented in Kotlin for Android—into a unified, safely compiled Rust core provides profound advantages in stability, deterministic execution, and testability. The strictness of Rust's type system, combined with its memory safety guarantees, eliminates entire categories of runtime errors, race conditions, and casting faults that frequently destabilize advanced mathematical operations and state management in virtual machine-based languages.

This comprehensive architectural report provides an exhaustive investigation into the optimal paradigms required to advance an existing Kotlin-Rust hybrid application into a robust, cross-platform ecosystem supporting both Android and iOS. It delivers a critical analysis of the proposed architectural constraints—specifically, the desire to maintain synchronous Rust execution while introducing timer-driven background operations—and resolves these contradictions through unidirectional message-passing state machines. Furthermore, the analysis addresses the critical challenge of continuous background execution on iOS, specifically focusing on Bluetooth Low Energy (BLE) biometric data ingestion within Apple's restrictive background execution limits, debunking the misconception of the "20-minute window." Finally, the report dictates the orchestration of Custom Artificial Intelligence developer tools, including the precise sequencing of Visual Studio Code Copilot instructions, to automate the translation of Kotlin models into Rust structures supported by UniFFI.

Architectural Critique and the Evolution of the Hybrid Model

The proposed architecture relies on synchronous, pass-by-value API calls initiated by Kotlin and executed within the Rust core using UniFFI bindings. While this approach has proven effective for stateless, computationally intensive tasks such as Fast Fourier Transforms (FFTs) and digital signal processing, expanding this model to encompass application state, database synchronization, and BLE telemetry necessitates a rigorous architectural critique.

The primary architectural contradiction within the proposed approach lies in the concurrency model. The requirement dictates that the system should not deal with threads or asynchronous

execution within Rust, relying entirely on synchronous calls initiated by Kotlin. Simultaneously, there is a requirement for Rust to manage application state and periodically push data to the database, the web, or over BLE via a "timer in Rust" that initiates a UI change or server contact. A strictly synchronous, threadless Rust core cannot autonomously wake up to execute a timer loop or asynchronously push data back into the Kotlin layer without a driving native thread. If Rust were to implement an internal timer, it would require spawning a background thread (std::thread::spawn) or integrating an asynchronous runtime like Tokio, which violates the stated constraint against overcomplicating the threading model.

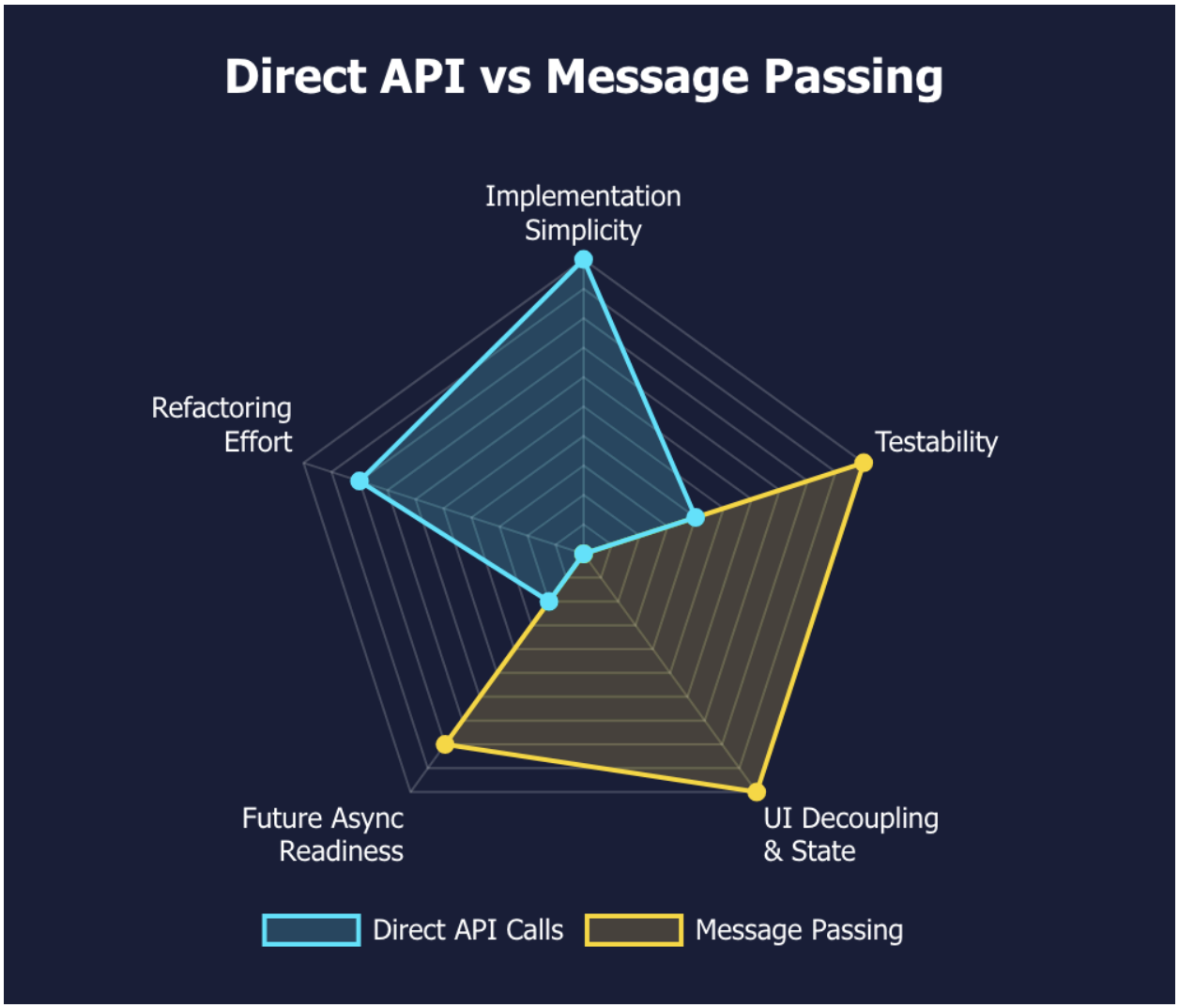
To resolve this contradiction while maintaining a purely synchronous Rust core, the architecture must adopt an "Inverted Control" pattern driven by the native shell. In this paradigm, the native operating system (Android or iOS) retains total ownership of all timers, threads, and asynchronous I/O scheduling. If a periodic database synchronization is required, the native shell initializes a platform-optimized timer (e.g., java.util.Timer in Kotlin or DispatchSourceTimer in Swift). Upon each tick, the native shell invokes a synchronous FFI call into the Rust core, passing a TimerTicked event. The Rust core processes this event and synchronously returns a declarative instruction—an Effect—that commands the native shell to execute the database write. This fully satisfies the constraint of keeping Rust synchronous and thread-free while achieving the desired state-driven periodic behavior.

State Machine Architecture for Deterministic Testing

The desire to implement state machines within Rust heavily complements the inverted control pattern. By modeling the Rust core as a deterministic, purely functional state machine, testing becomes exceptionally rigorous. In this architecture, the entire application state is held within a central Rust struct. State transitions only occur when the native shell passes strongly typed enum events over the UniFFI boundary.

Because the Rust core is insulated from the non-deterministic nature of network latency, thread interleaving, and database locking, integration testing does not require mocking complex OS-level interfaces. A developer can simulate an entire day of biometric data collection by passing a sequence of programmatic events into the state machine and asserting that the resulting state and output commands match the expected mathematical outcomes. This level of test-driven testability ensures that bugs are caught at compile-time or during isolated unit testing, long before the code is deployed to an emulator or physical device.

Foreign Function Interface Paradigms: API Calls vs. Message Passing



When migrating stateful business logic into a shared core, systems architects generally choose between direct synchronous API calls and unidirectional message passing. Both approaches are supported by UniFFI, but they present vastly different scaling characteristics as the application complexity increases.

The Direct Synchronous API Model

In a direct API model, the native shell invokes specific Rust functions tailored to individual tasks, such as analyze_accelerometer_data(data: [u8]) -> StatisticalModel. This is the current operational paradigm.

The advantage of this approach is its conceptual simplicity. It requires minimal boilerplate and aligns with traditional object-oriented programming mentalities. However, as the application scales to include database orchestration, web synchronization, and continuous BLE background ingestion, this model suffers from the "fat interface" anti-pattern. The UniFFI boundary becomes cluttered with hundreds of distinct function endpoints. Furthermore, if a deep statistical analysis determines that the user interface needs to update, passing that trigger back up the call stack through nested direct API calls becomes brittle and difficult to

trace.

The Unidirectional Message Passing Architecture

Message-passing architectures, heavily inspired by the Elm architecture and implemented in cross-platform Rust frameworks like Crux, provide a superior alternative for stateful mobile applications.[1]

In this paradigm, the UniFFI boundary is narrowed to a single primary entry point: process_event(event: AppEvent) -> Vec<AppEffect>.

- **Events (Inputs):** Strongly typed Rust enums representing everything that can possibly happen in the application (e.g., BleDataReceived([u8]), SyncTimerTicked, UserPressedSave).
- **Effects (Outputs):** Strongly typed Rust enums representing side-effects that the native shell must execute (e.g., WriteToDatabase(Query), UpdateUi(ViewModel), TransmitBluetooth(Payload)).

When the native shell invokes process_event, the Rust state machine updates its internal data models and synchronously returns a list of Effects.[1] The native shell iterates over these effects and executes them using platform-optimized asynchronous coroutines or threads.

The following table delineates the comparative advantages of these paradigms for the specific constraints of the project:

Architectural Parameter	Direct Synchronous API Calls	Unidirectional Message Passing
FFI Surface Area	Broad. Requires exposing numerous individual functions and data models via UniFFI macros.	Extremely narrow. Requires exposing only the Event and Effect enums and the core processor function.[1]
State Synchronization	Risk of state drift if multiple native threads invoke different mutating APIs simultaneously.	Native shell serializes events into a single queue, eliminating race conditions within the logic core.
Testability	High for pure math. Difficult for stateful sequences requiring simulated database reads.	Exceptional. Complex user journeys can be validated entirely in Rust without Native code.
UI Lifecycle Management	Native UI must manually poll the Rust core for updated graph data after calculations.	Rust explicitly returns an UpdateUi(GraphModel) effect when calculations conclude, cleanly decoupling logic from presentation.
Asynchronous I/O	Requires complex UniFFI callback interfaces (Foreign Traits) to handle network	Responses from native network calls are simply fed back into Rust as a new

	responses.[2]	NetworkResponseReceived event.[1]

For a cross-platform application managing biometric telemetry and sophisticated signal processing, the unidirectional message-passing approach is unequivocally recommended. It seamlessly integrates with the requested constraint of keeping Rust synchronous and delegating I/O prioritization to Kotlin's coroutines and Swift's Grand Central Dispatch.

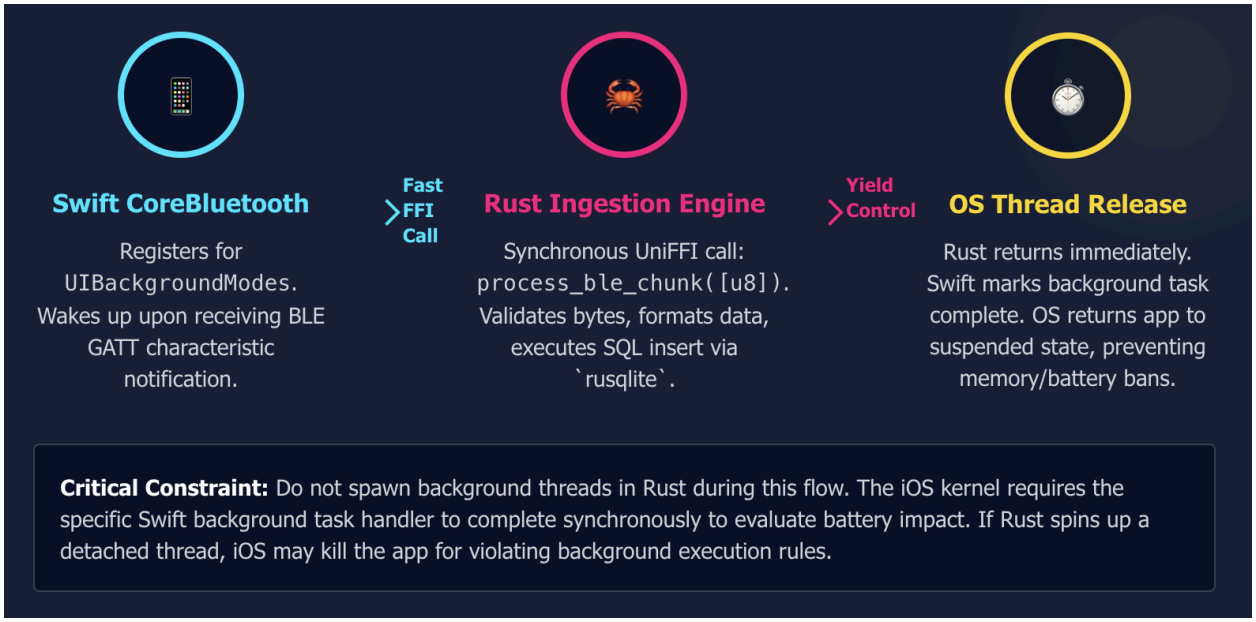
Overcoming iOS Background Execution and Biometric Ingestion Limitations

The most formidable architectural disparity between Android and iOS lies in background execution semantics. The existing Android application relies on a Foreground Service to maintain persistent CPU activity, ensuring continuous polling, calculation, and synchronization of biometric data. Apple's iOS explicitly prohibits this form of unbounded background execution, prioritizing stringent power management and user privacy.[3]

The assertion that iOS provides a "narrow CPU activity window every 20min or so" represents a critical conflation of two entirely separate iOS background APIs: BGAppRefreshTask and CoreBluetooth Background Execution.

Background Fetch (BGAppRefreshTask) is governed by an opaque machine-learning algorithm that grants the application execution time based on user habits, battery level, and network conditions. This can result in execution windows separated by 20 minutes, 2 hours, or several days.[4] This API is fundamentally useless for real-time biometric data collection, as the hardware peripheral's buffer will catastrophically overrun between fetch intervals.

Event-Driven CoreBluetooth Wake Cycles



To achieve continuous operation on iOS without risking App Store rejection or excessive battery drain, the application must completely invert its telemetry flow. The smartphone can no

longer dictate the polling schedule; instead, the custom BLE device firmware must orchestrate the data flow.

When the UIBackgroundModes array in the iOS Info.plist includes the bluetooth-central key, iOS permits the application to be woken from a deeply suspended state whenever a connected peripheral explicitly transmits an updated characteristic value via GATT Notifications or Indications.[5]

The lifecycle mechanics are precise and highly restricted:

1. The iOS application connects to the custom BLE peripheral and explicitly subscribes to the raw PPG signal characteristic using setNotifyValue(true, for: characteristic).[6]
2. When the user sends the application to the background, iOS almost immediately suspends the application process. All CPU execution halts, and native timers are paused.
3. The BLE peripheral accumulates biometric data into its hardware buffer. Before the buffer overruns, the firmware initiates a GATT Notification containing the encoded payload.
4. The iOS Bluetooth daemon (bluetoothd) intercepts the radio packet at the OS level and wakes the suspended application process in the background.[5]
5. The peripheral(_:didUpdateValueFor:error:) delegate method fires within the Swift shell.[6]

From the exact millisecond this delegate fires, iOS grants the application a strict execution window—typically averaging **10 seconds**—to process the data before aggressively suspending the application again.[7]

Optimizing the 10-Second Execution Window

Within this fleeting 10-second window, the Swift shell must extract the byte array, format it as a UniFFI input event, and pass it synchronously into the Rust core. The Rust core performs digital signal processing, updates its internal statistical models, and returns an Effect commanding the Swift shell to persist the summarized data to the local SQLite database.

If the DSP calculations or database writes threaten to exceed this 10-second limit, the operating system watchdog will abruptly terminate the application, generating a CPU_FATAL crash log.[8] To mitigate this risk, the Swift layer must explicitly request additional processing time the moment the wake event occurs by invoking UIApplication.shared.beginBackgroundTask(withExpirationHandler:).[7] This API temporarily halts the suspension timer, extending the execution window up to approximately 30 seconds.

Crucially, once the Rust core acknowledges that the statistical analysis is complete and the data is safely persisted, the Swift shell must immediately call UIApplication.shared.endBackgroundTask(). Promptly returning the CPU to sleep is a strict requirement; failing to yield the processor rapidly will result in the OS penalizing the application's energy score and potentially refusing future background wake requests.[8]

Mitigating Buffer Overruns via Firmware Coordination

Because the smartphone operates on a restricted sleep schedule, preventing data loss requires precise coordination between the iOS FFI architecture and the custom BLE device firmware.

The firmware must dictate a Connection Interval that balances high-throughput data transfer

against the requirement to let the iOS CPU sleep. A deeply short interval prevents the iOS device from sleeping entirely, triggering background termination for excessive battery drain. A long interval risks hardware buffer overruns. The firmware should dynamically adjust the Connection Interval, utilizing the longest possible duration that safely clears the sensor buffers before capacity is reached.

Furthermore, any commands the Rust core needs to push back to the BLE device during the 10-second wake window must utilize the WriteWithoutResponse GATT procedure.[10] Standard write requests require an acknowledgment from the peripheral, which introduces latency and blocks the Swift thread, risking task expiration. WriteWithoutResponse ensures the command is instantly queued in the OS Bluetooth stack, allowing the application to immediately return to sleep.

State Preservation and Restoration Resilience

Even with optimal background task management, the iOS operating system reserves the right to fully terminate the application if the device comes under extreme memory pressure (e.g., the user opens a memory-intensive camera application or high-fidelity game).[5] If the application is terminated, it will no longer receive BLE wake events, resulting in a total loss of all subsequent biometric data.

To counter this inevitable scenario, the application must implement CoreBluetooth State Preservation and Restoration.[11] By instantiating the CBCentralManager with a unique restoration identifier (e.g., CBCentralManagerOptionRestoreIdentifierKey), iOS assumes responsibility for monitoring the connected peripheral at the hardware level on behalf of the terminated application.[11]

If the peripheral subsequently sends a biometric notification, iOS will launch the application entirely in the background, bypassing the UI layer.[11] The willRestoreState delegate method fires, allowing the Swift shell to seamlessly re-instantiate the Rust core, reconstruct the state machine from the SQLite database, and process the incoming telemetry without requiring any user intervention.[12]

Advanced Subsystems: Push-to-Talk, Speech Recognition, and Localization

The integration of advanced subsystems such as Push-to-Talk (PTT), text-to-speech (TTS), and localization requires careful orchestration within the message-passing FFI boundary to maintain cross-platform parity.

Push-to-Talk and Audio Processing

Historically, background audio transmission on iOS required maintaining persistent Voice-over-IP (VoIP) sockets, a practice Apple has heavily restricted.[9] For the greenfield Swift application, integration with the native PushToTalk framework, introduced in iOS 16, is a strict technical requirement to avoid App Store rejection.[14] This framework utilizes the PTChannelManager to provide native system UI controls, enabling the transmission and receipt

of audio even when the application is suspended.[15]
Speech recognition and audio processing must be strategically partitioned. The native shells (Android Foreground Service and iOS PTT Framework) handle microphone access and raw buffer collection. If the architecture requires custom DSP on the audio stream (e.g., noise suppression or proprietary encoding), passing continuous, high-frequency raw audio buffers over the UniFFI boundary can introduce measurable overhead, as UniFFI performs memory allocation and copying during data transfer.
To optimize this, the native shells must aggregate the audio into optimally sized chunks before submitting a RawAudioChunkReceived([u8]) event to the Rust state machine. The Rust core processes the audio and returns a TransmitEncodedAudio([u8]) effect, commanding the native shell to broadcast the payload over the network. If native OS-level speech recognition APIs (like Android's SpeechRecognizer or iOS's SFSpeechRecognizer) are utilized, the transcription strings are simply passed into Rust as SpeechTranscribed(String) events, completely decoupling the platform-specific AI execution from the core business logic.

Localization and Web Interactions

Localization and string formatting should fundamentally reside in the native UI layers to leverage the extensive tooling built into Android Studio and Xcode. The Rust core should not hardcode localized strings. Instead, the Rust state machine should return formatting tokens or enumeration keys within its Effects or ViewModels. For example, if a mathematical calculation fails, Rust returns a DisplayError(ErrorKey::SensorCalibrationFailed) effect. The native Kotlin or Swift UI intercepts this key and maps it to the respective strings.xml or Localizable.strings file, ensuring platform-appropriate formatting for dates, currencies, and right-to-left languages.
Web interactions and database queries must follow the inverted control pattern. The Rust core emits a SubmitPayloadToServer(WebPayload) effect. The native layer, utilizing Kotlin's Retrofit or Swift's URLSession, executes the network request using platform-optimized thread pools and I/O prioritization. Upon completion, the native layer invokes the Rust interface with a ServerResponseReceived(Result) event, allowing the state machine to transition accordingly.

AI-Driven Development: Setting up VSCode Copilot, Agents, and Skills

Implementing a test-driven AI pipeline in VSCode using Copilot or custom agents to systematically port Kotlin models to Rust.

Step 1: Specification

Update `SPEC.md` detailing the Kotlin class to be migrated. Define the UniFFI exposed struct/enum, expected error types, and business logic mapping.

Step 2: Test Definition

Write Rust Unit tests based on the SPEC. Verify they fail. Update Kotlin UI to rely on a stubbed version of the new Rust interface.

Step 3: Execute / torust

Run the AI prompt against the Kotlin file. AI generates the Rust equivalent, updates Uniffi macros, and removes the old Kotlin logic.

Migrating a complex Kotlin codebase into a unified Rust core requires significant manual refactoring. However, the deployment of custom Artificial Intelligence developer tools—specifically utilizing GitHub Copilot extensions within Visual Studio Code—can dramatically accelerate this process by intelligently automating the translation of data models and business logic.[16]

Establishing the Monorepo Structure

Before implementing AI skills, the monorepo directory structure must be strictly defined to prevent dependency conflicts and ensure cross-platform build pipeline stability.[18] The repository must maintain absolute logical segregation:

Directory	Primary Function	Build Tooling
/android	Kotlin Jetpack Compose UI, Android foreground services, Room/SQLite.	Gradle
/ios	Greenfield SwiftUI, CoreBluetooth delegates, PTT Framework integrations.	Xcode / Swift Package Manager
/rust/stat3/core	Cargo workspace containing the pure state machine, DSP algorithms, and data models.	Cargo
/rust/stat3/android	Specific bindings and FFI configurations for Android deployment.	Cargo
/rust/stat3/ios	Specific bindings and FFI configurations for iOS deployment.	Cargo
/scripts	Shell scripts for generating XCFrameworks and Kotlin AARs using UniFFI bindgen.	Bash / Zsh

Sequence of AI Initialization Instructions

To ensure the LLM understands the architectural constraints of the project, workspace-level instructions must be established before generating code. VSCode automatically detects .github/copilot-instructions.md and AGENTS.md files in the repository root, seamlessly applying them to all chat requests.[19]

The sequence for setting up the AI environment is as follows:

Step 1: Define Global Architecture Rules (.github/copilot-instructions.md)

This file enforces the strict boundaries of the hybrid system, preventing the AI from hallucinating asynchronous Rust code or generating incorrect FFI bindings. The developer must create this file and populate it with the following markdown text:

Project Architecture and Cross-Platform Guidelines

FFI and Rust Core Rules

- All shared business logic resides in the /rust/stat3 module.
- Do not use asynchronous Rust (async/await), tokio, or spawn threads (std::thread). All Rust functions and state machines must remain strictly synchronous.
- Use UniFFI procedural macros (#, #[derive(uniffi::Enum)], #[uniffi::export]). Do not generate .udl interface files.
- Ensure all Rust structs mapping to Kotlin/Swift data classes implement Clone, Debug, and uniffi::Record.
- When translating Kotlin sealed classes, use Rust enum structures annotated with #[derive(uniffi::Enum)].
- The system uses a unidirectional message-passing architecture. Rust accepts Event enums and returns Effect enums.
- Map exceptions to Rust Result<T, E> where E is an enum implementing std::error::Error and #[derive(uniffi::Error)].

Native Shell Rules (Kotlin/Swift)

- When extracting logic from Kotlin to Rust, replace the original Kotlin function body with a direct invocation of the UniFFI generated Rust interface.
- Preserve Kotlin coroutine context and Swift GCD for all I/O operations.
- Do not move Android/iOS OS interactions (SQLite cursors, BLE GATT connections, PTT APIs) into Rust; only move the formatting, validation, and mathematical transformation logic.

Step 2: Initialize Specification Enforcement (AGENTS.md)

This file dictates how the AI interacts with the SPEC.md requirement files during test-driven

development.

Agent Directives

System Prompt Configuration for VSCode Agent

Provide this context to your LLM workspace settings or custom Copilot instructions:

```
"You are an expert systems engineer migrating an Android Kotlin codebase to a Rust
workspace (`/rust/stat3`) exposed via UniFFI. When instructed to migrate a
component:"
1. Analyze the provided Kotlin file/function.
2. Generate the equivalent Rust `struct` or `enum` inside `/rust/stat3/src/android/`.
3. Decorate the Rust structures with `#[derive(uniffi::Record)]` or
   `#[derive(uniffi::Enum)]` as appropriate.
4. Generate the business logic implementation using strictly typed Rust. Prefer
   returning `Result<T, ErrorEnum>` mapped to UniFFI.
5. Do not introduce async/await or threading. Keep functions synchronous.
6. Provide the refactored Kotlin stub that removes the internal logic and instead
   calls the newly generated UniFFI Rust bindings.
```

- Before writing any implementation code in Rust, Kotlin, or Swift, you MUST verify that the local SPEC.md file defines the behavior.
- If the feature is missing from SPEC.md, you must draft an update to the spec and request developer approval.
- Follow strict Test-Driven Development (TDD). Generate the Rust unit tests simulating the state machine event sequences, ensure they fail, and then implement the Rust logic.

Implementing the "/torust" Custom Chat Participant

Example Execution Instructions

```
/torust AccelerometerModel.kt

Target Action: Read `AccelerometerModel.kt`. Look at `SPEC.md` for context. Generate
`accelerometer_model.rs` with `#[derive(uniffi::Record)]`. Port the
`analyzeData(data: FloatArray)` function to a Rust `#[uniffi::export]` function.
Replace the Kotlin `analyzeData` body with the generated UniFFI call. Run `generate-
kotlin-bindings.sh`.
```

With the core instructions established, a custom VSCode Chat Participant extension can be created to provide the /torust skill. This participant intercepts user commands, appending necessary context regarding the selected Kotlin file and invoking the LLM to generate the exact Rust equivalent.[20]

The developer initiates the creation of the extension using the @workspace participant, supplying the following LLM prompt to Copilot Chat:
"@workspace Create a new VSCode Chat Participant extension named 'torust'. The extension must register a command '/torust' in package.json. When invoked, the request handler must read the currently active Kotlin file in the editor. It must extract the AST or raw text and submit it to the LLM with a system prompt instructing it to translate the Kotlin data model or function into a UniFFI-compliant Rust struct, enum, or synchronous state machine transition. It must also output the refactored Kotlin stub that invokes the new Rust FFI binding. Format the output in a split markdown code block."
When executed by the developer (e.g., /torust AccelerometerModel.kt), the AI dynamically reads the Kotlin data class, recognizes the types (e.g., DoubleArray, Long), and outputs the identical Rust # pub struct AccelerometerModel {... }, while stripping the business logic from the Kotlin file and replacing it with an FFI call.

Additional Recommended AI Skills

To accelerate the Greenfield iOS deployment and robust FFI testing, the following custom skills should be developed via VSCode Chat Participants:

AI Skill Command	Target Functionality	Operational Mechanic
/swiftui_sync	Automates UI duplication across platforms.	Reads an active Jetpack Compose Kotlin file, maps the UI modifiers and view models to the equivalent SwiftUI syntax, and wires the actions to the shared Rust Event enums.
/mock_ffi	Enhances TDD in the native shells.	Reads the Rust Effect enumerations and generates mock Kotlin/Swift executors that simulate database latency and network failures for native UI testing.
/gen_spec	Automates documentation synchronization.	Reads a newly generated Rust module and updates the adjacent SPEC.md to reflect the new state machine transitions, satisfying the AGENTS.md directive.

iOS Greenfield Initialization and Cross-Platform Management

Creating a greenfield SwiftUI application that exactly mirrors the Kotlin Jetpack Compose UI,

while sharing the same underlying logic, relies entirely on the successful deployment of the UniFFI bindings as an Apple XCFramework.[21]

The XCFramework Build Pipeline

The current Android workflow utilizes a shell script to build the Rust library and inject it into the Android directory. This same paradigm is easily adaptable for iOS without migrating to monolithic build systems.

A new shell script (generate-swift-bindings.sh) must be created to execute the following pipeline:

1. **Multi-Architecture Compilation:** Invoke cargo build --release --target aarch64-apple-ios (for physical iPhones) and cargo build --release --target aarch64-apple-ios-sim (for iOS Simulators).[21]
2. **Header Generation:** Execute the uniffi-bindgen CLI tool to parse the compiled .dylib library, generating the Swift wrapper file (stat3.swift) and the bridging C-header (stat3.h).[21]
3. **Framework Assembly:** Utilize the xcodebuild -create-xcframework command line tool to bundle the generated .a static libraries and headers into a unified Stat3Core.xcframework.[21]
4. **Integration:** The script moves the generated XCFramework and the .swift wrapper file into the Xcode project directory.

The Swift application developer then imports the framework, and the generated Swift structures directly mirror the Kotlin classes.

Duplicating Kotlin UI and Managing Regressions

When establishing the SwiftUI app from nothing, the developer utilizes the previously defined /swiftui_sync AI skill. Because the UI for both platforms is treated as a "dumb renderer" reacting to Rust Effects, the Swift translation is heavily procedural. ViewModel fields in Jetpack Compose directly translate to @State or @ObservedObject properties in SwiftUI, seamlessly reacting to the data payloads provided by the FFI boundary.

When new functions are required or bug reports are filed, the triage process becomes deterministic:

1. **Isolate the Domain:** Determine if the bug relates to visual rendering (e.g., text clipping, animation stutter) or business logic (e.g., incorrect math, missed state transition).
2. **Platform vs. Core Resolution:** Visual bugs are addressed exclusively in the respective native UI layer. Logic bugs are reproduced by adding a failing test to the Rust state machine simulator, modifying the Rust code to pass the test, and triggering the build scripts. The fix is immediately inherited by both the iOS and Android applications, ensuring absolute functional parity.

Industry Case Studies: Lessons Learned and Architectural Pitfalls

The deployment of a shared core wrapped in native UI has been successfully executed by several high-profile organizations. Analyzing their post-mortems reveals distinct patterns of success and failure that directly apply to this project.[22]

Successful Implementations

1Password: Faced with the challenge of supporting complex cryptographic synchronization across Windows, Mac, iOS, and Android, 1Password transitioned to a shared Rust core architecture.[23] They pushed the use of Rust beyond standard algorithmic computation, utilizing it to handle complex navigation and synchronization logic.[24] By routing UI states through a unified core, they achieved nearly 80% code reuse.[24] A crucial lesson learned was the importance of standardizing the core's interfaces, allowing frontend developers to build platform-specific UIs without needing to understand the underlying cryptographic execution.[25]

Mozilla Firefox Sync: Mozilla, the creators of UniFFI, utilized the tool to deploy Rust within mobile browsers.[26] They discovered that utilizing UniFFI allowed them to avoid manually writing JNI wrappers for Kotlin and Objective-C bridges for Swift, drastically accelerating deployment timelines.[26] Their success underlines the viability of UniFFI for high-performance, production-grade applications serving hundreds of millions of users.[27]

TigerBeetle: While a distributed financial database rather than a mobile application, TigerBeetle's architecture demonstrates the extreme power of pure state machines. By explicitly modeling all network, clock, and storage faults within deterministic state machines, they achieve complete logical availability and testability.[28] This validates the proposed approach of testing the Rust mobile core by simulating sequences of events independently of the host operating system.

Failures and Architectural Pitfalls

The GoMobile Failure: Teams attempting similar shared-core architectures using the Go language (via GoMobile) routinely report severe friction.[29] The primary failures stem from limited type support across the FFI boundary (e.g., inability to pass complex objects or slices easily), forcing reliance on bloated Protobuf serialization, and massive binary sizes due to the inclusion of the Go garbage collector.[29] Rust avoids these pitfalls entirely; UniFFI flawlessly handles complex structs, and Rust's lack of a garbage collector keeps memory overhead negligible and binary sizes minimal.

The "Chatty" FFI Bottleneck: A widespread failure mode in shared-core architectures is designing an overly "chatty" interface. Developers often attempt to model the Rust core exactly like a native object-oriented framework, exposing thousands of fine-grained getter and setter methods. Because every call across the FFI boundary incurs a minor serialization and context-switching cost, looping over a Rust array from Swift by calling a get_item_at_index() function hundreds of times per second catastrophically degrades performance.

The essential lesson from these failures is that the FFI boundary must be designed around coarse-grained data transfer. When Kotlin or Swift requests data, the Rust state machine must serialize the entire view model state and pass it across the boundary in a single, comprehensive

operation.

Strategic Conclusions

The strategic intent to transition from a Kotlin-heavy architecture to a unified Rust core flanked by native Jetpack Compose and SwiftUI shells represents a highly resilient, modern engineering paradigm. By pushing stateful logic, mathematical transformations, and biometric data validation into a strict, compile-time verified environment, the application drastically reduces logic duplication and cross-platform drift.

To execute this architecture successfully, the system must rigidly adhere to a unidirectional, message-passing interface. By discarding the notion of asynchronous Rust threads and instead relying on the native shells to manage timers, database scheduling, and OS-level interactions, the Rust core remains entirely synchronous, deterministic, and highly testable.

Addressing continuous background execution on iOS demands an abandonment of the Android-style continuous polling loop. The application must leverage CoreBluetooth's event-driven wake cycles, meticulously coordinating peripheral firmware buffers with the OS-mandated 10-second background execution windows, utilizing beginBackgroundTask and state restoration to guarantee absolute data persistence.

Finally, by implementing custom VSCode Copilot instructions and participants that explicitly enforce UniFFI formatting, architectural boundaries, and test-driven specifications, the manual overhead of translating Kotlin data models into Rust can be minimized. This enables a phased, automated, and strictly governed migration, hardening the core business logic while empowering the native platforms to deliver fluid, flawless user interfaces.

Works cited

1. Overview - Crux: Cross-platform app development in Rust - GitHub Pages, accessed April 18, 2026, https://redbadger.github.io/crux/
2. Yes, you can do that. UniFFI allows you to generate Kotlin and Swift wrappers, w... | Hacker News, accessed April 18, 2026, https://news.ycombinator.com/item?id=37073422
3. Configuring background execution modes | Apple Developer Documentation, accessed April 18, 2026, https://developer.apple.com/documentation/xcode/configuring-background-execution-modes
4. iOS Background Execution Limits: What Every Developer Must Know (2026) - AppsOnAir, accessed April 18, 2026, https://www.appsonair.com/blogs/background-execution-limits-in-ios-what-every-developer-must-know
5. Core Bluetooth Background Processing for iOS Apps - Apple Developer, accessed April 18, 2026, https://developer.apple.com/library/archive/documentation/NetworkingInternetWeb/Conceptual/CoreBluetooth_concepts/CoreBluetoothBackgroundProcessingForIOSApps/PerformingTasksWhileYourAppIsInTheBackground.html
6. Common Challenges in Integrating Bluetooth Low Energy (BLE) in iOS with Swift -

Medium, accessed April 18, 2026, https://medium.com/@premajanoti/common-challenges-in-integrating-bluetooth-low-energy-ble-in-ios-with-swift-63918460090e
7. CoreBluetooth background execution 10sec? - Stack Overflow, accessed April 18, 2026, https://stackoverflow.com/questions/38968325/corebluetooth-background-execution-10sec
8. Energy Efficiency Guide for iOS Apps: Work Less in the Background - Apple Developer, accessed April 18, 2026, https://developer.apple.com/library/archive/documentation/Performance/Conceptual/EnergyGuide-iOS/WorkLessInTheBackground.html
9. Energy Efficiency Guide for iOS Apps: Fundamental Concepts - Apple Developer, accessed April 18, 2026, https://developer.apple.com/library/archive/documentation/Performance/Conceptual/EnergyGuide-iOS/FundamentalConcepts.html
10. Apple's IOS Core Bluetooth: The Ultimate Guide - Punch Through, accessed April 18, 2026, https://punchthrough.com/core-bluetooth-guide/
11. Core Bluetooth and backgrounding: Detection of a device and triggering an action, even after being days in background mode? - Stack Overflow, accessed April 18, 2026, https://stackoverflow.com/questions/15062902/core-bluetooth-and-backgrounding-detection-of-a-device-and-triggering-an-action
12. What does it mean for background processing of Bluetooth data to be 'session-based' in iOS? - Stack Overflow, accessed April 18, 2026, https://stackoverflow.com/questions/47759848/what-does-it-mean-for-background-processing-of-bluetooth-data-to-be-session-bas
13. iOS Silent Push Limits - by Shobhakar Tiwari - Medium, accessed April 18, 2026, https://medium.com/@shobhakartiwari/ios-silent-push-limits-7d0c65b642f4
14. What is Apple's Push to Talk Framework? - The Zello Blog, accessed April 18, 2026, https://blog.zello.com/what-to-know-about-apples-ptt-framework-and-zello?hsLang=en
15. Creating a Push to Talk app | Apple Developer Documentation, accessed April 18, 2026, https://developer.apple.com/documentation/pushtotalk/creating-a-push-to-talk-app
16. Chat overview - Visual Studio Code, accessed April 18, 2026, https://code.visualstudio.com/docs/copilot/chat/copilot-chat
17. How to Customize Visual Studio Code Chat with GitHub Copilot and Semantic Kernel, accessed April 18, 2026, https://techcommunity.microsoft.com/blog/educatordeveloperblog/how-to-customize-visual-studio-code-chat-with-github-copilot-and-semantic-kernel/4367996
18. 7 Mobile App Architecture Trends We Tested (3 Failed Hard) - Wix Studio, accessed April 18, 2026, https://backlinksindiit.wixstudio.com/app-development-expe/post/7-mobile-app-

architecture-trends-tested-3-failed-why
19. Use custom instructions in VS Code, accessed April 18, 2026, https://code.visualstudio.com/docs/copilot/customization/custom-instructions
20. Build a code tutorial chat participant with the Chat API - Visual Studio Code, accessed April 18, 2026, https://code.visualstudio.com/api/extension-guides/ai/chat-tutorial
21. Building an iOS App with Rust Using UniFFI - DEV Community, accessed April 18, 2026, https://dev.to/almaju/building-an-ios-app-with-rust-using-uniffi-200a
22. They Rewrote Mobile In Rust — What Paid Off, What Hurt, What Stayed (From Public Case Studies) | by The Stateless Samurai | Medium, accessed April 18, 2026, https://medium.com/@samurai.stateless.coder/they-rewrote-mobile-in-rust-what-paid-off-what-hurt-what-stayed-from-public-case-studies-827b792c0b2e
23. Member Spotlight: 1Password - The Rust Foundation, accessed April 18, 2026, https://rustfoundation.org/media/member-spotlight-1password/
24. Next-generation Proton Mail mobile apps: more than meets the eye, accessed April 18, 2026, https://proton.me/blog/next-generation-proton-mail-mobile-apps
25. We're the creators of 1Password for Linux. Ask us anything! - Reddit, accessed April 18, 2026, https://www.reddit.com/r/1Password/comments/ng9psn/were_the_creators_of_1password_for_linux_ask_us/
26. mozilla/uniffi-rs: a multi-language bindings generator for rust - GitHub, accessed April 18, 2026, https://github.com/mozilla/uniffi-rs
27. Introducing Uniffi for React Native: Rust-Powered Turbo Modules - Mozilla Hacks, accessed April 18, 2026, https://hacks.mozilla.org/2024/12/introducing-uniffi-for-react-native-rust-powered-turbo-modules/
28. TigerBeetle, accessed April 18, 2026, https://tigerbeetle.com/
29. Considering replacing GoMobile with Rust uniffi for shared core mobile/desktop/core/wasm, accessed April 18, 2026, https://www.reddit.com/r/rust/comments/1l6jp8e/considering_replacing_gomobile_with_rust_uniffi/